Trabajo Práctico Final: Desarrollo de un Sistema de Gestión

1. Contexto y Enunciado

El objetivo de este trabajo práctico es diseñar y desarrollar una aplicación de escritorio para digitalizar la administración de un dominio o negocio a elección del grupo (por ejemplo: gestión de biblioteca, control de inventario de una tienda, administración de un gimnasio, sistema de reservas de hotel, etc.).

La solución debe construirse bajo el paradigma de **Programación Orientada a Objetos (POO)**, contar con una interfaz gráfica basada en **formularios** y almacenar la información de forma persistente en una base de datos **MySQL**.

2. Requisitos Técnicos

Arquitectura y Diseño Orientado a Objetos

- **Separación en Capas:** El proyecto debe estructurarse obligatoriamente en al menos 3 capas lógicas:
    - **Capa de Presentación (GUI):** Formularios, controles e interacciones visuales.
    - **Capa de Lógica de Negocio (BLL / Dominio):** Clases que representen las entidades del problema elegido.
    - **Capa de Acceso a Datos (DAL / DAO):** Clases exclusivas para la conexión y ejecución de operaciones contra MySQL, implementando patrones.
- **Conceptos Fundamentales de POO:**
    - **Encapsulamiento:** Atributos de clase privados, exponiendo su estado únicamente mediante métodos de acceso (Getters/Setters o Propiedades).
    - **Herencia y Polimorfismo:** Definición de al menos una jerarquía de herencia coherente con el dominio elegido (ejemplo: una clase base `Persona` heredada por `Cliente` y `Empleado`, o `Articulo` heredada por `Libro` y `Revista`).
    - **Manejo de Excepciones:** Bloques de control estructurado (`try-catch`) para gestionar errores de conexión a la base de datos, conversiones de tipos de datos y validaciones de interfaz.

Base de Datos (MySQL)

- Diseño e implementación de una base de datos relacional acorde a la temática.

- La base de datos debe contener **un mínimo de 3 tablas relacionadas**, utilizando correctamente claves primarias (PK) y claves foráneas (FK). Deberá existir al menos una relación de 1 a N o de N a M.
- Uso estricto de **Stored Procedure, Trigger y Transacciones**.

Interfaz Gráfica (Formularios)

- **Pantalla Principal (Menú):** Contenedor principal que permita navegar hacia los distintos módulos de la aplicación.
- **Formularios de Mantenimiento (ABM/CRUD):**
    - Controles de entrada de datos (campos de texto, calendarios/fechas, casillas de verificación) con validación de campos obligatorios o formatos específicos antes de enviar a la base de datos.
    - Una vista de grilla o tabla (*DataGrid, JTable, etc.*) que muestre el listado de registros actualizados en tiempo real.
- **Controles Dinámicos:** Utilización de listas desplegables (*ComboBox*) cuya información provenga dinámicamente de la base de datos para manejar las claves foráneas (ejemplo: seleccionar una Categoría al crear un Producto).

3. Requisitos Funcionales

1. **Gestión de Entidades Principales:** Alta, modificación, consulta y eliminación (física o lógica) de al menos dos entidades maestras.
2. **Mapeo Objeto-Relacional Manual:** Toda consulta proveniente de MySQL debe instanciarse en objetos y colecciones de las clases de dominio antes de ser enviada a la interfaz gráfica. La interfaz gráfica no debe leer clases nativas de datos (como `ResultSet` o `DataTable`) directamente.
3. **Búsqueda y Filtros:** Implementación de un buscador en tiempo real en las grillas de datos (por ejemplo, filtrar por nombre, código o documento).
4. **Operación Relacional/Transaccional:** Un módulo que relacione entidades y genere un registro de negocio. (Ejemplo: Agendar un turno médico a un paciente, registrar una venta de artículos a un cliente, registrar el préstamo de un libro a un socio).

4. Entregables

1. **Código Fuente Completo:** Repositorio o archivo comprimido con la estructura del proyecto (excluyendo carpetas de binarios compilados o dependencias temporales).
2. **Script SQL (`script_bd.sql`):** Archivo que contenga la creación de la base de datos, creación de tablas, definición de restricciones (PK/FK) y sentencias `INSERT` con datos de prueba (*seeders*).
3. **Documentación del Diseño:**
    - Diagrama de Clases (UML) reflejando la estructura de la capa de dominio.
    - Diagrama Entidad-Relación (DER) de la base de datos.
4. Aplicación de escritorio corriendo sin errores en tiempo de ejecución.
5. Presentación.